

King Saud University College of Computer and Information Sciences Computer Science Department	
CSC340 Programming Languages & Compilation	First Semester 1447 - 2025

CSC340 Project README File

Section# 44260

Student Name	ID
Leena Alonayq	444200942
Shahad Aldamegh	444201207
Amal Almaliki	444201022
Renad Aljarallah	444201212

1. Grammers

statement -> error

program -> PROGRAM IDENT SEMICOLON declarations BEGIN_PROGRAM statements
END_PROGRAM

declarations -> epsilon

| declaration SEMICOLON declarations

declaration -> id_list COLON type

| id_list COLON ARRAY L_PAREN NUMBER R_PAREN OF type

id_list -> IDENT

| id_list COMMA IDENT

type -> INTEGER

statements -> epsilon

| statement

| statement SEMICOLON statements

statement -> assignment

| if_stmt

| while_stmt

| io_stmt

| error

assignment -> IDENT ASSIGN expression

| IDENT L_PAREN expression R_PAREN ASSIGN expression

if_stmt -> IF condition THEN statements ENDIF

| IF condition THEN statements ELSE statements ENDIF

while_stmt -> WHILE condition LOOP statements ENDLOOP

io_stmt -> READ var_list

| WRITE var_list

var_list -> var

| var_list COMMA var

var -> IDENT

| IDENT L_PAREN expression R_PAREN

condition -> cond_or

cond_or -> cond_and

| cond_or OR cond_and

cond_and -> rel

| cond_and AND rel

rel -> expression

| expression EQ expression

| expression NEQ expression

| expression LT expression

| expression GT expression

| expression LTE expression

| expression GTE expression

| NOT rel

| TRUE

| FALSE

| L_PAREN condition R_PAREN

expression -> expression ADD expression

| expression SUB expression

| expression MULT expression

| expression DIV expression

| L_PAREN expression R_PAREN

| IDENT

| IDENT L_PAREN expression R_PAREN

| NUMBER

| SUB expression

1. Sample Runs

1- Outputs for valid inputs

Input

```
-- Requires 5 integer arguments, all of which are assumed to be < 100
program mytest;
n : integer;
i, j, k : integer;
x, y, z : integer;
t : array (100) of integer;
beginprogram -- main program
read n, i, j, x, y;
if i<j and j<=n and i>=0 then
  t(i) := i;
  t(j) := j;
  k := t(i);
  t(i) := t(j);
  t(j) := t(i);
else
  while x > y or false loop
    k := (1+y - x)*n / 3 + x;
    x := x - 10;
  write k;
  endloop;
endif;
write i, j, x, y, t(i), t(j);
endprogram
```

Output

```

id_list -> IDENT
type -> INTEGER
declaration -> id_list COLON type
id_list -> IDENT
id_list -> id_list COMMA IDENT
id_list -> id_list COMMA IDENT
type -> INTEGER
declaration -> id_list COLON type
id_list -> IDENT
id_list -> id_list COMMA IDENT
id_list -> id_list COMMA IDENT
type -> INTEGER
declaration -> id_list COLON type
id_list -> IDENT
type -> INTEGER
declaration -> id_list COLON ARRAY L_PAREN NUMBER R_PAREN OF type
declarations -> empty
declarations -> declaration SEMICOLON declarations
declarations -> declaration SEMICOLON declarations
declarations -> declaration SEMICOLON declarations
declarations -> declaration SEMICOLON declarations
var -> IDENT
var_list -> var
var -> IDENT
var_list -> var_list COMMA var
var -> IDENT
var_list -> var_list COMMA var
var -> IDENT
var_list -> var_list COMMA var
var -> IDENT
var_list -> var_list COMMA var
io_stmt -> READ var_list
statement -> io_stmt
expression -> IDENT
expression -> IDENT
rel -> expression LT expression
cond_and -> rel
expression -> IDENT
expression -> IDENT
rel -> expression LTE expression
cond_and -> cond_and AND rel
expression -> IDENT
expression -> NUMBER
rel -> expression GTE expression
cond_and -> cond_and AND rel
cond_or -> cond_and
condition -> cond_or
expression -> IDENT
expression -> IDENT
assignment -> IDENT L_PAREN expression R_PAREN ASSIGN expression
statement -> assignment
expression -> IDENT
expression -> IDENT
assignment -> IDENT L_PAREN expression R_PAREN ASSIGN expression
statement -> assignment
expression -> IDENT

```

```

expression -> IDENT L_PAREN expression R_PAREN
assignment -> IDENT ASSIGN expression
Result of expression = false
statement -> assignment
expression -> IDENT
expression -> IDENT
expression -> IDENT L_PAREN expression R_PAREN
assignment -> IDENT L_PAREN expression R_PAREN ASSIGN expression
statement -> assignment
expression -> IDENT
expression -> IDENT
expression -> IDENT L_PAREN expression R_PAREN
assignment -> IDENT L_PAREN expression R_PAREN ASSIGN expression
statement -> assignment
statements -> empty
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
expression -> IDENT
expression -> IDENT
rel -> expression GT expression
cond_and -> rel
cond_or -> cond_and
rel -> FALSE
cond_and -> rel
cond_or -> cond_or OR cond_and
condition -> cond_or
expression -> NUMBER
expression -> IDENT
expression -> expression ADD expression
expression -> IDENT
expression -> expression SUB expression
expression -> L_PAREN expression R_PAREN
expression -> IDENT
expression -> expression MULT expression
expression -> NUMBER
expression -> expression DIV expression
expression -> IDENT
expression -> expression ADD expression
assignment -> IDENT ASSIGN expression
Result of expression = false
statement -> assignment
expression -> IDENT
expression -> NUMBER
expression -> expression SUB expression
assignment -> IDENT ASSIGN expression
Result of expression = -10
statement -> assignment
var -> IDENT
var_list -> var
io_stmt -> WRITE var_list
statement -> io_stmt
statements -> empty
statements -> statement SEMICOLON statements

statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
while_stmt -> WHILE condition LOOP statements ENDLOOP
statement -> while_stmt
statements -> empty
statements -> statement SEMICOLON statements
if_stmt -> IF condition THEN statements ELSE statements ENDIF
statement -> if_stmt
var -> IDENT
var_list -> var
var -> IDENT
var_list -> var_list COMMA var
var -> IDENT
var_list -> var_list COMMA var
var -> IDENT
var_list -> var_list COMMA var
expression -> IDENT
var -> IDENT L_PAREN expression R_PAREN
var_list -> var_list COMMA var
expression -> IDENT
var -> IDENT L_PAREN expression R_PAREN
var_list -> var_list COMMA var
io_stmt -> WRITE var_list
statement -> io_stmt
statements -> empty
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
program -> PROGRAM IDENT SEMICOLON declarations BEGIN_PROGRAM statements END_PROGRAM

```

2- Outputs for valid inputs

Input

```
program test;
beginprogram
x := 5;
y := x + 3;
write y;
endprogram
```

Output

```
declarations -> empty
expression -> NUMBER
assignment -> IDENT ASSIGN expression
Result of expression = 5
statement -> assignment
expression -> IDENT
expression -> NUMBER
expression -> expression ADD expression
assignment -> IDENT ASSIGN expression
Result of expression = 3
statement -> assignment
var -> IDENT
var_list -> var
io_stmt -> WRITE var_list
statement -> io_stmt
statements -> empty
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
program -> PROGRAM IDENT SEMICOLON declarations BEGIN_PROGRAM statements END_PROGRAM
```

3- Outputs for Invalid inputs with lexical error

Input

```
program lexDemo;
beginprogram
1x := 5 + 3;      -- invalid: starts with number
_y := 4 * 2;      -- invalid: starts with underscore
z_ := 10 / 2;     -- invalid: ends with underscore
endprogram
```

Output

```
declarations -> empty
Error at line 3, column 1: identifier must begin with a letter "1x"
expression -> NUMBER
expression -> NUMBER
expression -> expression ADD expression
assignment -> IDENT ASSIGN expression
Result of expression = 8
statement -> assignment
Error at line 4, column 1: identifier must begin with a letter "_y"
expression -> NUMBER
expression -> NUMBER
expression -> expression MULT expression
assignment -> IDENT ASSIGN expression
Result of expression = 8
statement -> assignment
Error at line 5, column 1: identifier cannot end with an underscore "z_"
expression -> NUMBER
expression -> NUMBER
expression -> expression DIV expression
assignment -> IDENT ASSIGN expression
Result of expression = 5
statement -> assignment
statements -> empty
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
statements -> statement SEMICOLON statements
program -> PROGRAM IDENT SEMICOLON declarations BEGIN_PROGRAM statements END_PROGRAM
```

Note: We assumed that even lexical errors should not cause the program to crash.

4- Outputs for Invalid inputs with syntax error

Input

```
program calcTest;
beginprogram
x := 5 + ;           -- Missing right-hand side of addition

if i<j and then      -- Missing condition after and
y := 4 / 2 ;
endif;               -- Missing else

endprogram
```

Output

```
declarations -> empty
expression -> NUMBER
Syntax error at line 3: syntax error
Syntax error at line 3: invalid statement
statements -> statement
Syntax error at line 5: syntax error
Syntax error at line 6: invalid statement
statements -> statement
Syntax error at line 7: syntax error
Syntax error at line 7: invalid statement
statements -> statement
program -> PROGRAM IDENT SEMICOLON declarations BEGIN_PROGRAM statements END_PROGRAM
```